

# Genetic Algorithms

— A Practical Introduction —



Leonardo Chieco

# Genetic Algorithms: A Practical Introduction

The objective of this guide is to provide a concise yet informative overview of the fascinating world of genetic algorithms, showing how principles inspired by natural evolution can be applied to solve complex computational problems.

Genetic algorithms were first introduced by John Holland in the 1970s, who formalized the idea of using evolutionary principles to explore large and complex solution spaces. Since then, GAs have grown into a powerful tool in computer science, engineering, optimization, and artificial intelligence.

Over the years, they have been applied to a wide range of real-world problems, from designing antennas and scheduling airline crews to evolving game strategies and optimizing neural network architectures.

In this tutorial, we will explore the core concepts of GAs, demonstrate how they work through practical examples, and provide visual, step-by-step illustrations so that you can intuitively understand both the theory and the practical power of this bio-inspired approach to problem-solving.

Enjoy the reading!

## The Author

Leonardo Chieco is an electronic engineer with over 20 years of experience in the design and development of automation control software (PC/PLC), electronic boards for industrial applications, firmware, robotics, and mechatronics.

LinkedIn: <https://www.linkedin.com/in/leonardo-chieco-53550b129/>

# 1. Introduction to Genetic Algorithms

Genetic Algorithms (GAs) are a class of optimization and search algorithms inspired by the principles of natural evolution and genetics. They belong to the broader family of evolutionary algorithms, which mimic biological processes to solve complex problems that are difficult to tackle using traditional analytical or deterministic methods.

At their core, Genetic Algorithms simulate the process of natural selection, where the fittest individuals are more likely to survive and reproduce. By iteratively selecting, combining, and mutating candidate solutions, GAs evolve populations toward better solutions over successive generations.

The main components of a Genetic Algorithm include:

- Population: A set of candidate solutions, often represented as vectors or strings.
- Fitness function: A measure of how well each individual solves the problem.
- Selection: The process of choosing individuals to reproduce based on their fitness that is, the ability to solve the problem.
- Crossover (Recombination): Combining two parent solutions to produce offspring or child.
- Mutation: Randomly altering a child to maintain diversity in the population.
- Iteration (Generations): Repeating selection, crossover, and mutation until a satisfactory solution is found.

Genetic Algorithms are directly inspired by Charles Darwin's theory of evolution by natural selection. In nature:

- Individuals with advantageous traits have higher chances of surviving and reproducing.
- Offspring inherit traits from their parents, sometimes with small variations.
- Over time, populations adapt to their environment, improving their overall fitness.

Similarly, GAs “evolve” solutions by favoring better-performing candidates and introducing diversity through mutation and recombination.

Genetic Algorithms are highly versatile and have been applied in a wide range of fields, including:

- Engineering and Design: Optimizing structures, mechanical components, or circuit layouts.
- Artificial Intelligence: Tuning neural networks, reinforcement learning policies, or game strategies.
- Operations Research: Solving scheduling, routing, and resource allocation problems.
- Bioinformatics: Finding optimal DNA sequence alignments or protein structures.
- Finance: Portfolio optimization, algorithmic trading, and risk management.
- Control Systems: Designing controllers and tuning parameters in complex dynamical systems.
- Automatically tune a Kalman filter’s parameters (like process and measurement noise covariances) by evolving candidate solutions to minimize estimation error over generations.

One of the main strengths of GAs is their ability to explore large and complex search spaces where traditional gradient-based or exhaustive methods fail. While they may not always guarantee the global optimum, they often provide good approximate solutions efficiently.

## 2. How Genetic Algorithms Work

Genetic Algorithms (GAs) are inspired by the way nature evolves species over time. They are search and optimization techniques that iteratively improve a set of candidate solutions to a problem, imitating the mechanisms of survival of the fittest, reproduction, and variation.

The process begins with a population of solutions, which can be represented in various forms, for example, arrays of numbers, strings, or other structures, depending on the problem.

Each solution is evaluated using a fitness function, which quantifies how well it solves the task at hand. The higher the fitness, the closer the solution is to the desired outcome.

Once the fitness of each individual is known, the algorithm selects certain solutions to act as parents for the next generation. Those that perform better have a higher chance of being chosen, but weaker solutions are not entirely discarded, preserving diversity and allowing exploration of the search space.

Recombination, or crossover, is the next key step. Two parent solutions are combined to produce one or more offspring, often by exchanging parts of their structure. This allows beneficial traits from different solutions to combine and potentially produce superior candidates.

To prevent the population from becoming too uniform and to explore new possibilities, mutation is introduced. Mutation makes small, random modifications to some individuals, ensuring that the algorithm does not get stuck in suboptimal solutions.

After offspring are created and optionally mutated, they form the next generation. Elitism is often applied, meaning that the best-performing solutions from the previous generation are directly carried over to the next generation, preserving progress and ensuring that high-quality solutions are not lost.

Depending on the implementation, the algorithm may keep some of the best solutions from the previous generation (elitism) to preserve progress.

This cycle of selection → crossover → mutation → replacement repeats over many generations. The algorithm stops when a pre-defined condition is met, for example, a maximum number of generations, or when a solution reaches an acceptable fitness level.

## Summary of Key Steps

- Encoding Solutions: Represent each candidate in a structured format suitable for the problem.
- Initial Population: Generate a set of random solutions.
- Fitness Evaluation: Assign a quality score to each solution.
- Selection: Choose individuals to reproduce based on their fitness.
- Crossover: Combine traits from parents to produce offspring.
- Mutation: Introduce small random changes to maintain diversity.
- Elitism: Preserve the best candidate of a population.
- Replacement: Form the new generation, possibly keeping top performers.
- Termination: Repeat until a satisfactory solution is found or a stopping condition occurs.

By continuously applying these steps, Genetic Algorithms are capable of exploring complex, multidimensional search spaces and finding good approximations to solutions that might be difficult or impossible to compute with traditional methods. Their strength lies not in guaranteeing a perfect answer, but in efficiently discovering strong solutions even in challenging problems.

### 3. Genetic Algorithms in Action

Genetic Algorithms (GAs) are not just a metaphorical way to solve problems, they are grounded in solid principles of search and optimization that make them useful for exploring vast, complicated solution spaces that confound traditional techniques.

Traditional numerical methods rely on local information, for example, how a small change in a variable affects the objective. This works well when the landscape is smooth and convex. But many real-world problems are highly non-linear, discontinuous, with many local optima, landscapes that traditional methods tend to get “stuck” in.

GAs tackle this issue by:

- Exploring multiple candidate solutions simultaneously, rather than following a single path.
- Balancing exploration and exploitation: exploration through mutation and random variation; exploitation through selection of better solutions.
- Not requiring derivative information, so they can optimize functions that don't have gradients or are noisy.

This allows GAs to navigate complex, rugged search spaces more effectively than many classical algorithms.

#### Step-by-Step example

To make Genetic Algorithms more tangible, let's consider a simple practical problem: solving a  $3 \times 3$  linear system of equations.

First, let's give some definitions.

In a Genetic Algorithm, a **chromosome** represents a complete candidate solution to the problem.

In our case the problem is solving a 3-variable linear system:

$$\begin{cases} a_{1,1}x + a_{1,2}y + a_{1,3}z = b_1 \\ a_{2,1}x + a_{2,2}y + a_{2,3}z = b_2 \\ a_{3,1}x + a_{3,2}y + a_{3,3}z = b_3 \end{cases}$$

A candidate solution consists of a set of values for the three variables x, y, z.

Thus, each chromosome in the GA is:

$$[x \quad y \quad z]$$

Each represents a complete possible solution to the linear system.

Genes are the fundamental units of information within a chromosome. They are the elements that can mutate or combine during crossover.

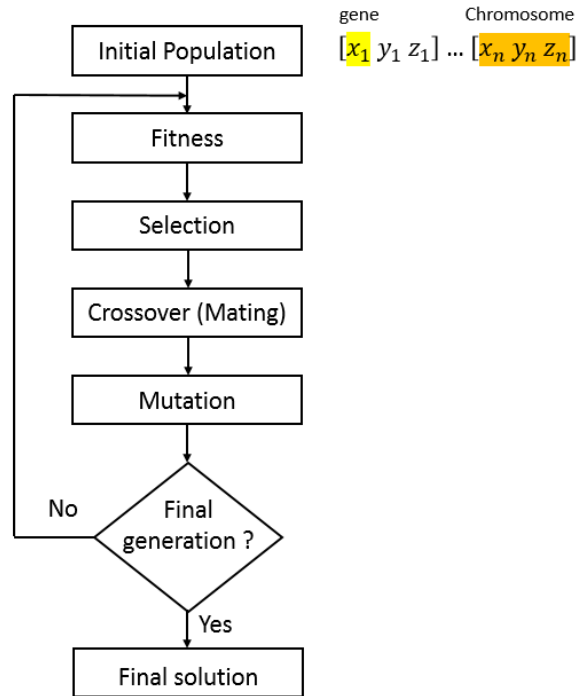
In our example the chromosome  $[x \quad y \quad z]$  has 3 genes: x, y and z.

Each gene is a numeric value contributing to the quality (fitness) of the solution.

During crossover, some genes are exchanged between parent chromosomes to create new offspring (child).

During mutation, individual genes randomly change to introduce variability.





Now, suppose the system is:

$$\begin{cases} 2x + 1y + 3z = 9 \\ 1x + 2y + 2z = 8 \\ 3x + 2y + 1z = 7 \end{cases}$$

The goal is to find values of  $x$ ,  $y$ , and  $z$  that satisfy all three equations.

## Step 1: Representing Solutions

Each candidate solution is represented as a vector of three numbers:

$$[x \ y \ z]$$

For example, an initial random population might look like:

$$[1 \ 2 \ 3], [0 \ 1 \ 2], [3 \ 2 \ 1], [2 \ 0 \ 1], [1 \ 1 \ 1], \dots$$

Each represents a different initial guess at the solution. The algorithm evaluates all these simultaneously, not one at a time and this is crucial for robustness.

## Step 2: Evaluating Fitness

We assign a fitness score to each chromosome based on how well it solves the problem. Think of fitness as a measure of usefulness. The closer a solution is to optimal, the higher its score.

Let's define:

$$Err = \sum_{j=1}^3 |a_{j,1}x_i + a_{j,2}y_i + a_{j,3}z_i - b_j|$$

And then convert error into fitness:

$$Fitness = \frac{1}{1 + Err}$$

In this case:

$$Fitness ([x \ y \ z]) = \frac{1}{1 + |2x + 1y + 3z - 9| + |1x + 2y + 2z - 8| + |3x + 2y + 1z - 7|}$$

The fitness function is designed so that minimizing the error directly maximizes fitness: when the error is close to zero, fitness approaches its maximum value of 1, whereas large errors result in fitness values close to zero.

Mathematically:  $Fitness \in (0, 1]$

In case of perfect solution, the error is zero and fitness = 1

This formulation turns a minimization problem (error) into a maximization problem (fitness), which is standard in GAs.

Example calculation for  $[1 \ 2 \ 3]$ :

$$\text{Equation 1: } 2 \times 1 + 1 \times 2 + 3 \times 3 = 2 + 2 + 9 = 13 \rightarrow \text{Error} = |13 - 9| = 4$$

$$\text{Equation 2: } 1 \times 1 + 2 \times 2 + 2 \times 3 = 1 + 4 + 6 = 11 \rightarrow \text{Error} = |11 - 8| = 3$$

$$\text{Equation 3: } 3 \times 1 + 2 \times 2 + 1 \times 3 = 3 + 4 + 3 = 10 \rightarrow \text{Error} = |10 - 7| = 3$$

$$\text{Total error} = 4 + 3 + 3 = 10 \rightarrow \text{Fitness} = \frac{1}{1 + \text{Err}} = \frac{1}{1 + 10} = 0.0909$$

### Step 3: Selecting Parents

Basic rule: better solutions have a higher chance of being selected for reproduction.

Suppose after evaluating fitness, the solutions are ranked as:

|         |    |   |    |                |                                                                                    |
|---------|----|---|----|----------------|------------------------------------------------------------------------------------|
| Rank 1: | [2 | 1 | 1] | Fitness = 0.25 |  |
| Rank 2: | [1 | 2 | 3] | Fitness = 0.09 |                                                                                    |
| Rank 3: | [3 | 0 | 1] | Fitness = 0.08 |                                                                                    |
| Rank 4: | [0 | 1 | 2] | Fitness = 0.06 |                                                                                    |
| Rank 5: | [1 | 1 | 1] | Fitness = 0.05 |                                                                                    |

Better chromosomes are more likely to reproduce.

We might select parents probabilistically, favoring higher ranks:

Parent 1 → [2 1 1]  
Parent 2 → [1 2 3]

In principle, better candidates are more likely to be parents, but weaker ones can still contribute diversity. However, GAs don't just choose the best, they choose based on probability.

Imagine a lottery where the number of tickets equals fitness.

This helps keep the population diverse and avoids early stagnation.

### Step 4: Crossover (Combining parents and creating new variants)

Crossover recombines two parent genes to produce offspring.

For example, the average of the parental genes can be calculated, promoting stability and gradual convergence: the offspring tend to remain within the region of the search space already identified as promising, rather than abruptly switching to unrelated solutions. This encourages the exploitation of good solutions, while still allowing for the subsequent introduction of diversity through mutation.

Example:

$$\begin{array}{l} \text{Parent A: } [2 \quad -4 \quad 6] \\ \text{Parent B: } [4 \quad 0 \quad 2] \end{array} \rightarrow \text{Child: } \left[ \frac{2+4}{2} \quad \frac{-4+0}{2} \quad \frac{6+2}{2} \right] = [3 \quad -2 \quad 4]$$

This allows offspring to inherit and mix traits from both parents, similar to genetic inheritance in nature.

In some applications, elitism is used, that is, a strategy in which the best chromosomes are passed unchanged to the next generation. This ensures that the highest-quality solutions are preserved.

### **Step 5: Mutation (Adding variation and maintaining diversity)**

Mutation introduces small, random changes, preventing the population from becoming too similar.

So, to explore new possibilities, we randomly mutate some genes. Typically, a certain mutation rate is applied (e.g., 20%). For example:

Offspring 2 before mutation:  $[2.0 \quad 2.0 \quad 3.0]$

Offspring 2 after mutation:  $[2.0 \quad 2.0 \quad 3.1]$  ← z changed from 3 to 3.1  
(mutation in 3rd gene)

Offspring 1 before mutation:  $[1 \quad 1 \quad 1]$

Offspring 1 after mutation:  $[0.9 \quad 1 \quad 1]$  ← x changed from 1 to 0.9 (mutation in 1st gene)

Mutation allows the GAs to explore new regions of the search space, helping it escape local optima.

### **Step 6: Form Next Generation (Replace and repeat)**

Once offspring are created, selection and reproduction repeat. Over successive generations, the population gradually improves.

With each generation, the average fitness tends to increase because:

- Good solutions survive and reproduce.
- Bad solutions rarely propagate their traits.

This mimics the accumulation of beneficial adaptations observed in biological evolution.

Genetic Algorithms work not because of a single clever trick, but because they combine systematic search with randomness and adaptation, mirroring natural evolution. They are especially effective when:

- The problem space is large, complex, and poorly understood.
- Traditional optimization fails due to local optima or missing mathematical structure.
- Parallel evaluation and search are possible.

Although they don't guarantee perfect solutions, GAs often provide very good approximate solutions in contexts where classical algorithms struggle.

In Appendix 1, you'll find this example coded in Python.

At each run, the software generates a new system and calculates its exact solution. It then runs the genetic algorithm. The software output is:

System of equations:

$$3x + 2y + 4z = -22$$

$$4x + 7y + 9z = -49$$

$$5x + 2y + 1z = -17$$

Exact solution:  $x=-2$ ,  $y=-2$ ,  $z=-3$

Generation 0, Fitness: 0.142857, Best:  $x=-3.0000$ ,  $y=0.0000$ ,  $z=-4.0000$

Generation 50, Fitness: 0.865584, Best:  $x=-2.0137$ ,  $y=-1.9507$ ,  $z=-3.0180$

Generation 100, Fitness: 0.951928, Best:  $x=-2.0045$ ,  $y=-2.0114$ ,  $z=-2.9884$

Generation 150, Fitness: 0.952195, Best:  $x=-2.0001$ ,  $y=-2.0214$ ,  $z=-2.9834$

Generation 200, Fitness: 0.952195, Best:  $x=-2.0001$ ,  $y=-2.0214$ ,  $z=-2.9834$

Generation 250, Fitness: 0.952195, Best:  $x=-2.0001$ ,  $y=-2.0214$ ,  $z=-2.9834$

Generation 300, Fitness: 0.952195, Best:  $x=-2.0001$ ,  $y=-2.0214$ ,  $z=-2.9834$

Generation 350, Fitness: 0.952342, Best:  $x=-1.9938$ ,  $y=-2.0269$ ,  $z=-2.9832$

Generation 400, Fitness: 0.981182, Best:  $x=-1.9968$ ,  $y=-2.0105$ ,  $z=-2.9937$

Generation 450, Fitness: 0.981182, Best:  $x=-1.9968$ ,  $y=-2.0105$ ,  $z=-2.9937$

Generation 500, Fitness: 0.981182, Best:  $x=-1.9968$ ,  $y=-2.0105$ ,  $z=-2.9937$

Generation 550, Fitness: 0.981182, Best:  $x=-1.9968$ ,  $y=-2.0105$ ,  $z=-2.9937$

Generation 600, Fitness: 0.981182, Best:  $x=-1.9968$ ,  $y=-2.0105$ ,  $z=-2.9937$

Generation 650, Fitness: 0.981182, Best:  $x=-1.9968$ ,  $y=-2.0105$ ,  $z=-2.9937$

Generation 700, Fitness: 0.981182, Best:  $x=-1.9968$ ,  $y=-2.0105$ ,  $z=-2.9937$

Generation 750, Fitness: 0.988148, Best:  $x=-1.9985$ ,  $y=-2.0009$ ,  $z=-3.0007$

Generation 800, Fitness: 0.988148, Best:  $x=-1.9985$ ,  $y=-2.0009$ ,  $z=-3.0007$

```
Generation 850, Fitness: 0.988148, Best: x=-1.9985, y=-2.0009, z=-3.0007
Generation 900, Fitness: 0.988148, Best: x=-1.9985, y=-2.0009, z=-3.0007
Generation 950, Fitness: 0.988148, Best: x=-1.9985, y=-2.0009, z=-3.0007
Generation 999, Fitness: 0.988148, Best: x=-1.9985, y=-2.0009, z=-3.0007
```

```
Final best solution:
x  $\approx$  -1.9985, y  $\approx$  -2.0009, z  $\approx$  -3.0007
Fitness: 0.988148
```

You can try changing the main algorithm parameters in this line to see what happens:

```
def genetic_algorithm(pop_size=400, generations=1000, mut_rate=0.3, mut_amp=1.0):
```

## 4. Conclusions

Genetic Algorithms represent a powerful and versatile approach to problem-solving, drawing inspiration directly from the principles of natural evolution. By maintaining a population of candidate solutions and iteratively applying selection, crossover, and mutation, GAs are able to explore complex solution spaces in ways that traditional, deterministic methods cannot.

Key points to take away:

- Population-based search allows simultaneous exploration of multiple potential solutions, reducing the risk of getting trapped in local optima.
- Fitness-driven selection ensures that better solutions propagate over generations, while mutation and crossover maintain diversity and adaptability.
- GAs do not require derivative information or a smooth search landscape, making them suitable for problems that are non-linear, discontinuous, or poorly understood.
- The combination of exploration and exploitation enables GAs to balance global search with local refinement, often finding very strong approximate solutions even in highly complex domains.
- Their flexibility and adaptiveness make them applicable across a wide range of fields: engineering, optimization, scheduling, machine learning, and beyond.

GAs illustrate how bio-inspired computation can effectively tackle optimization challenges that are difficult or impossible to solve with classical methods. While they may not always guarantee a perfect solution, they consistently provide robust, near-optimal results with the ability to adapt dynamically to changing conditions.

## 5. Appendix 1

```
# ----- SYSTEM -----
def rand_int(a, b):
    return random.randint(a, b)

def generate_system():
    true_sol = [rand_int(-5, 5) for _ in range(3)]
    A = []
    B = []
    for _ in range(3):
        row = [rand_int(1, 9) for _ in range(3)]
        A.append(row)
        B.append(sum(row[i] * true_sol[i] for i in range(3)))
    return A, B, true_sol

# ----- FITNESS -----
def fitness(ind, A, B):
    err = sum(abs(sum(A[i][j]*ind[j] for j in range(3)) - B[i]) for i in range(3))
    return 1 / (1 + err)

# ----- GA OPERATORS -----
def tournament(pop, A, B):
    a = random.choice(pop)
    b = random.choice(pop)
    return a if fitness(a, A, B) > fitness(b, A, B) else b

def crossover(a, b):
    return [(a[i] + b[i]) / 2 for i in range(3)]

def mutate(ind, rate, amp):
    return [v + (random.uniform(-amp, amp) if random.random() < rate else 0) for v in ind]

# ----- GA LOOP -----
def genetic_algorithm(pop_size=400, generations=1000, mut_rate=0.3, mut_amp=1.0):
    A, B, true_sol = generate_system()
    print("System of equations:")
    for i in range(3):
        print(f"{A[i][0]}x + {A[i][1]}y + {A[i][2]}z = {B[i]}")
    print(f"Exact solution: x={true_sol[0]}, y={true_sol[1]}, z={true_sol[2]}\n")

    # initialize population
    pop = [[rand_int(-10, 10) for _ in range(3)] for _ in range(pop_size)]

    history = []
    for gen in range(generations):
        pop.sort(key=lambda ind: fitness(ind, A, B), reverse=True)
        best = pop[0]
        history.append(fitness(best, A, B))

        if gen % 50 == 0 or gen == generations - 1:
            print(f"Generation {gen}, Fitness: {fitness(best, A, B):.6f}, Best:
x={best[0]:.4f}, y={best[1]:.4f}, z={best[2]:.4f}")

        next_gen = [best] # elitism: keep best
        while len(next_gen) < pop_size:
            parent1 = tournament(pop, A, B)
            parent2 = tournament(pop, A, B)
            child = crossover(parent1, parent2)
            child = mutate(child, mut_rate, mut_amp)
            next_gen.append(child)
        pop = next_gen
```



```
print("\nFinal best solution:")
best = pop[0]
print(f"x ≈ {best[0]:.4f}, y ≈ {best[1]:.4f}, z ≈ {best[2]:.4f}")
print(f"Fitness: {fitness(best, A, B):.6f}")

# ----- RUN -----
if __name__ == "__main__":
    genetic_algorithm()
```